

END-TO-END SUCCESS RATE IS A BLUNT INSTRUMENT: PROCESS-LEVEL ATTRIBUTION OF FAULT SENSITIVITY IN A WEB AGENT

Anonymous authors

Paper under double-blind review

ABSTRACT

Web agents are evaluated almost exclusively by end-to-end task success, and that number is routinely read as a measure of robustness. We test how far this reading survives a controlled fault-injection study. Using a LangGraph ReAct agent on WebArena with the official functional-correctness evaluator, we inject five non-adversarial faults (DOM omission, popup blocking, HTTP errors, timeouts, and tool parameter errors) in a paired control/fault design, and analyse 400 official evaluations alongside a 1302-trial behavioural record. Three findings emerge. First, end-to-end success is a blunt, high-variance instrument: with $n = 30\text{--}31$ native-evaluator pairs per fault and an observed discordance of 0.234, the paired design cannot reach 0.80 power at *any* effect size the data can express, and no fault is significant after Holm correction (all $p_{\text{Holm}} = 1.000$). Detecting a 10 pp difference — the smallest we would consider practically interesting — needs on the order of 181 pairs per fault, several times more than we have. Second, the same faults are sharply visible one level down: HTTP errors add +4.22 ReAct steps and +149.9 s of wall-clock time per pair ($p < 0.001$), even though their success-rate effect is indistinguishable from zero. Third, the measuring instrument is itself a confound: a compatibility fallback in the evaluator path fails by construction on all 76 trials it handles, moving the frozen baseline’s headline rate from 85.0% to 52.1% depending only on whether those trials are stratified or pooled. We conclude that robustness claims for web agents should rest on paired, process-level measurements under an explicitly stratified evaluator, and we give a staged design that would make the model-versus-architecture attribution question answerable.

1 INTRODUCTION

Web agents — language models that drive a real browser through a ReAct-style loop of reasoning, tool calls, and page observations — are now benchmarked almost exclusively by one number: the fraction of tasks whose final environment state satisfies the benchmark’s functional evaluator (Zhou et al., 2024; Koh et al., 2024). That number is then compared across models, across prompting strategies, and — increasingly — across fault-injection conditions, on the implicit assumption that a small drop in success rate means the agent is robust and a large drop means it is fragile.

This paper asks whether that assumption holds. We take a single agent configuration (a LangGraph ReAct agent, 9 WebArena tools, an accessibility-tree observation channel) on the WebArena shopping, Reddit, and GitLab sites, and subject it to a paired fault-injection experiment: for each of five non-adversarial faults, every (task, seed) cell is run twice, once clean and once with the fault injected at a fixed step, and both runs are scored by the official evaluator. The pairing is what makes the design informative — it removes task difficulty and per-seed variance from the comparison, and it lets us run an exact McNemar test rather than comparing two independent proportions.

The headline result is negative, and that is the point. After Holm correction, *no fault has a detectable effect on official success* ($p_{\text{Holm}} = 1.000$ for all five). The single largest point estimate, -12.9 pp for tool parameter errors, is comfortably inside a bootstrap interval that spans zero. Read the usual way,

this would be reported as “the agent is robust to these faults.” We show that reading is not supported: with only 30–31 native-evaluator pairs per fault, the design has essentially no power, and the honest statement is that the experiment could not have detected a moderate effect had one existed.

Where the faults *are* visible is one level below the task outcome. In a 1302-trial behavioural record that trades the evaluator for step and wall-clock instrumentation, HTTP errors add +4.22 ReAct steps and +149.9 s per pair, and tool parameter errors add +2.30 steps — both far beyond noise. End-to-end success is blind to these effects; process-level measurement is not. This is the paper’s central methodological claim: for web agents, task success is a blunt and high-variance instrument, and robustness conclusions drawn from it alone are under-determined.

Two further results shape how such experiments should be run. First, the evaluator path matters more than any fault we inject: a compatibility fallback handling 76 of the 400 official evaluations fails by construction, and pooling rather than stratifying those trials moves the frozen baseline’s success rate from 85.0% to 52.1% — a 32.9 pp swing driven purely by instrumentation. Second, we document degeneracies in the auxiliary artifacts (a recovery table whose columns are constant or defined by the outcome it claims to explain; behavioural counts whose “1302 trials” are in fact 400 pairs), which are easy to introduce and, once in a pipeline, easy to read as evidence.

We make three contributions:

- **A paired, evaluator-stratified attribution protocol** for web-agent fault injection, with exact and mid- p McNemar tests, pair-unit bootstrap intervals, Wilcoxon/Hodges–Lehmann tests on process deltas, and explicit multiplicity control (section 3).
- **An empirical demonstration that the standard outcome measure is underpowered** at realistic sample sizes, quantified with minimum detectable effects and the sample sizes that would actually be needed (section 5).
- **Evidence that the measurement instrument can dominate the effect being measured**, together with a stratified analysis that removes the confound, and a staged design that would make the model-versus-architecture question answerable (section 7).

We are explicit about scope: the data cover one model and one architecture, so we make *no* claim about how much of the robustness gap is attributable to the model versus the architecture, which was the motivating question. section 7 states what the present design can and cannot identify.

2 RELATED WORK

Web agent benchmarks. WebArena (Zhou et al., 2024) provides self-hosted replicas of four web applications with 812 long-horizon tasks and a functional evaluator that scores the resulting environment state rather than the action sequence. Its reported GPT-4 agent success rate of 14.41 % against a human 78.24 % established the long-horizon gap that motivates this line of work. VisualWebArena (Koh et al., 2024) adds visually grounded tasks and shows that a multimodal agent reaches only 16.4 % against a human 88.7 %. Both benchmark agent *capability* under clean conditions; neither injects faults, and both report a single success number per configuration.

Safety and prompt-injection evaluation. WASP (Evtimov et al., 2025) and SecureWebArena (Levy et al., 2025) treat the page as an adversarial surface, measuring intermediate attack success separately from end-to-end goal success and introducing process-level rates such as reasoning-compromise and payload-delivery. Their lesson — that a single outcome number cannot describe a multi-step agent’s failure modes — applies directly to non-adversarial settings, which is where our work sits.

Fault injection for tool-using agents. ChaosLLM (Zhang et al., 2024) interposes a middleware between an agent and its tools to inject unreachability, slowness, and error responses, and finds hangs the most damaging of the fault types it studies. WAREX (Zhang et al., 2025) uses a transparent HTTP proxy to inject network faults and popups into WebArena itself without modifying agent or benchmark, and reports success-rate drops above 70 % under network errors. ReliabilityBench (Gupta et al., 2025) combines repeatability, input perturbation, and infrastructure-fault tolerance into a single reliability surface and compares ReAct with Reflexion. These systems establish that

faults matter; our focus is different — we ask what the *measurement* can and cannot resolve, and we show that the paired design and the evaluator stratum, not the effect size alone, determine whether a conclusion is warranted.

Multi-agent fault attribution. MAS-FIRE (Wang et al., 2025) catalogues fifteen fault classes across planning, memory, reasoning, action, and communication, and injects them non-invasively into MetaGPT, CAMEL, and Table-Critic. It distinguishes fault detection, local repair, and final task success — a process-level decomposition we adopt in spirit but cannot replicate empirically here, because the recovery labels available in our artifacts are degenerate (section 5).

Gap. What is missing is a study that (i) injects non-adversarial faults into WebArena while keeping the *official* evaluator as the outcome, (ii) uses a paired design so that the comparison is not swamped by task heterogeneity, and (iii) reports the power of that design rather than interpreting its nulls as robustness. We provide exactly that, and the answer we get is methodological: the outcome measure is the bottleneck.

3 METHOD

3.1 AGENT AND ENVIRONMENT

The agent is a LangGraph state-graph ReAct loop (Yao et al., 2023; Team, 2024): `START` → `agent` → `tools` → `agent` → `...` → `END`. At each turn the agent receives a text rendering of the page’s accessibility tree and emits either a tool call or a final answer. The tool set comprises 9 WebArena actions (click, type, hover, scroll, go-back, go-forward, tab-focus, new-tab, and a no-op wait) dispatched over Playwright.

Two counter conventions matter for interpreting the cost columns later, so we fix them now. `step_count` counts *executor* ReAct turns only; planner and replanner invocations are counted in `llm_calls` but not in `steps`. In the ReAct configuration studied here the two coincide, but they diverge under plan-and-execute, which is one reason the architecture comparison in section 5 is reported as a pilot rather than a ranking.

3.2 FAULT MODEL

We inject five faults, chosen to span the observation, network, and action layers while remaining non-adversarial:

- **DOM omission** (`web_dom_missing`) — elements are dropped from the accessibility tree, so the agent observes a page that does not match the one it is acting on (observation layer).
- **Popup block** (`web_popup_block`) — a blocking overlay occludes actionable content (observation layer).
- **HTTP error** (`web_http_error`) — navigations return a server error (network layer).
- **Timeout** (`web_timeout`) — requests stall to the client deadline (network layer).
- **Parameter error** (`agent_param_error`) — a tool call is executed with a corrupted argument, so it fails at the action layer and the agent must recover.

Each fault is injected at a fixed step index through a seeded `FaultInjector`. The seed and the injection step are recorded per trial, so a fault condition is reproducible exactly. Injection intensity is held at high throughout. The implementation contains ten fault definitions in total; the five above are the ones with committed official evaluations, and we restrict every empirical claim in this paper to those five. The remaining five (targeting GitLab-specific state) are implemented but not part of the main result, and we do not claim anything about them.

Table 1: Committed experiment artifacts. Behavioral rows never enter an official-success denominator.

Dataset	Records	Design	Evaluation
Frozen ReAct control baseline	48	16 tasks $\times$ 3 seeds	official; 25 success
Behavioral fault matrix	800	5 faults $\times$ 16 tasks $\times$ 5 seeds $\times$ 2	no HAR; behavioral only
Official fault subset	400	5 faults $\times$ 8 tasks $\times$ 5 seeds $\times$ 2	official; 256 success
Architecture pilot	54	3 tasks $\times$ 3 seeds $\times$ 3 conditions $\times$ 2 arch	official; PILOT
Recovery subset	45	5 faults $\times$ 3 tasks $\times$ 3 seeds	log heuristics; subset of the official 400

3.3 PAIRED DESIGN AND ATTRIBUTION QUANTITIES

For a fault F and a (task, seed) cell u , let $Y_u^{(0)}, Y_u^{(1)} \in \{0, 1\}$ be official success in the control and fault arms. The design runs both arms of every cell, so the unit of analysis is the pair, and the estimand is the paired risk difference

$$D(F) = \mathbb{P}(Y^{(1)} = 1) - \mathbb{P}(Y^{(0)} = 1), \quad (1)$$

estimated by the mean of the within-pair differences $d_u = Y_u^{(1)} - Y_u^{(0)}$. Because $d_u \in \{-1, 0, +1\}$, the sample mean is exactly the difference of the two marginal rates on the paired subset, and its sign is determined by the two discordant counts b (control passes, fault fails) and c (fault passes, control fails). We therefore report b , c , and an exact McNemar test on them.

We deliberately do *not* define a “responsibility share” between model and architecture. Doing so requires a design in which both factors vary — Model $\times$ Architecture $\times$ Fault — and ours varies only the fault. A variance decomposition applied to these data would attribute to “model” a quantity that is constant by construction. section 7 specifies the design that would identify it.

3.4 STATISTICAL PROCEDURE

All tests are implemented in the Python standard library, with no SciPy dependency, and are unit-tested against hand-computed values (see the Reproducibility statement).

This section states the procedure in outline; Appendix A gives the full derivations, the seed-sensitivity envelope, and the power table.

The primary test is the two-sided *exact* McNemar test on the discordant counts, reported with its mid- p variant and a Haldane–Anscombe corrected matched odds ratio, and with Holm–Bonferroni multiplicity control across the five faults. Intervals for $D(F)$ are percentile bootstraps that resample *pair units* — the correct unit for a paired design — under a fixed seed, with a task-clustered sensitivity interval and a multi-seed envelope as checks. Process measures use the two-sided Wilcoxon signed-rank test with the Hodges–Lehmann shift estimator, since the deltas are heavy-tailed and tied. Power is obtained by solving the paired-proportion equation for δ at the *observed* discordance $\pi_d = 0.234$; because the risk difference is bounded by π_d rather than by 1, the search must be bracketed on $(0, \pi_d]$, and targets that are unreachable at any effect size are reported as *undefined* rather than as a number.

4 EXPERIMENTAL SETUP

4.1 ARTIFACTS, AND WHAT EACH MAY BE USED FOR

The evidence in this paper comes from five committed artifacts that are *not* interchangeable. Table 1 lists them; we describe the distinctions that matter, because conflating them is the single easiest way to produce a wrong number.

Three distinctions carry real weight. The **official fault subset** (400 evaluations) is the only source for success-rate claims, and it contains no step or timing columns. The **behavioural fault matrix** (1302 records) retained no HAR, so it carries no official verdict at all — every row is marked NOT_EVALUATED_NO_HAR — and it supports process-measure claims and nothing else; note also that its 1302 records are 400 pairs logged twice, not independent samples. The **official control baseline** (48 trials) is the only clean-condition artifact carrying both an official verdict and process

measures. The **architecture pilot** (54 rows) and **recovery subset** (45 rows) are small exploratory artifacts, treated as pilots throughout.

Two consequences we enforce mechanically rather than by convention. First, official success is never aggregated across artifacts: the behavioural matrix cannot enter a success-rate denominator, which is why the process results and the success results below are reported from different data and never combined into a single sentence about “cost per unit of success.” Second, the baseline and the official subset share a column schema but not a design (16 tasks $\times$ 3 seeds versus 8×5), so rows are tagged with their source artifact and the two are never concatenated.

4.2 THE EVALUATOR HAS TWO PATHS, AND THEY ARE NOT EQUIVALENT

The official evaluator in this stack has a native path and a compatibility fallback used when the native retrieval check cannot be applied. The fallback is *structurally* failure-biased: its null-retrieval schema branch returns success only when the state matches *and* both retrieved values are null, which our runs almost never satisfy. Empirically, of the 76 evaluations it processed in the official subset, **zero** succeeded, and the same pattern holds in the baseline (28.6% against 85.0% on the native path).

We therefore stratify every official result by evaluator path and treat the native stratum as primary. Table 2 shows the effect on the frozen baseline: the pooled rate is 52.1%, the native rate is 85.0%, a 32.9 pp gap produced entirely by which trials happened to be scored by which path. Reported without stratification, that gap would be attributed to the agent.

Because each arm of a pair may be scored by a different path, the native analysis restricts to pairs where *both* arms were scored natively. This is why the per-fault n drops from 5 seeds $\times$ 8 tasks = 40 to 30–31. We report both the pair-restricted rates (the quantity Δ and McNemar refer to) and the per-arm marginal native rates, and we label them separately in Table 3 because they are computed over different subsets and are not interchangeable.

4.3 METRICS

Success rate is the official evaluator’s verdict. *Paired risk difference* Δ is in percentage points, fault minus control, on the pair-restricted subset. *Process cost* is the within-pair delta in `step_count` and wall-clock seconds. *Discordance* π_d is the fraction of pairs in which the two arms disagree; it is the quantity that governs power, and it is reported rather than assumed.

4.4 REPRODUCIBILITY OF THE RUNS

Every trial records its fault seed, injection step, model profile, architecture, and git commit, so a fault condition is reproducible at the level of the design rather than the token sequence — the provider does not guarantee bit-identical sampling for a fixed seed. Wall-clock is host- and load-dependent, which is why process-cost claims use within-pair deltas only; Appendix E quantifies the drift.

5 RESULTS

5.1 THE EVALUATOR PATH MOVES THE HEADLINE NUMBER MORE THAN ANY FAULT

Table 2 stratifies the frozen control baseline by evaluator path. On the native path the agent succeeds in 85.0% of 20 trials; on the compatibility fallback it succeeds in 28.6% of 28. Every one of the fallback failures is structural — the fallback’s null-retrieval branch is unreachable for these tasks — not a property of the agent. Pooling the two paths yields 52.1%, and that pooled number is what a pipeline without stratification would report as “the baseline.”

This is a 32.9 pp artefact of instrumentation, and it is larger than any fault effect measured anywhere in this paper. Two rules follow, and we apply them throughout: never pool the paths, and never compare an arm scored on one path against an arm scored on the other.

Table 2: Frozen 48-trial ReAct baseline, stratified by evaluator path. Native and fallback rows are *not* comparable and must never be pooled; the pooled row is reported for completeness only.

Stratum	Trials	Success	Success rate	Mean steps	Mean time (s)
All trials	48	25	52.1%	6.40	122.9
Native evaluator	20	17	85.0%	6.30	91.0
Fallback evaluator	28	8	28.6%	6.46	145.7

Table 3: Primary fault result on **native-evaluator pairs only**. This is the table the paper relies on; the pooled version is contaminated by a fallback adapter that fails by construction. n is below 40 because a pair is dropped when either arm was scored by the fallback adapter.

Fault	n	Disc.	Control	Fault	Control [§]	Fault [§]	Δ (pp)	95% CI (pp)	p_{exact}	p_{mid}	p_{Holm}	OR
DOM omission	30	10	83.3%	76.7%	84.4%	78.8%	-6.7	[-26.7, +13.3]	0.754	0.549	1.000	0.69
Popup block	31	9	67.7%	77.4%	69.7%	77.4%	+9.7	[-9.7, +29.0]	0.508	0.344	1.000	1.86
HTTP error	31	8	71.0%	71.0%	71.9%	73.5%	+0.0	[-19.4, +16.1]	1.000	1.000	1.000	1.00
Timeout	31	3	77.4%	80.6%	78.8%	81.2%	+3.2	[-6.5, +12.9]	1.000	0.625	1.000	1.67
Parameter error	31	6	93.5%	80.6%	93.9%	80.6%	-12.9	[-29.0, +0.0]	0.219	0.125	1.000	0.27
All faults	154	36	78.6%	77.3%	79.8%	78.3%	-1.3	[-9.1, +6.5]	0.868	0.743	1.000	0.90

Δ is the paired risk difference (fault - control); the interval is a percentile bootstrap over pair units and is descriptive — the exact McNemar p is the primary inferential quantity. OR is the Haldane-corrected matched odds ratio. [§]Per-arm *marginal* native rate, whose denominator is every native trial in that arm and therefore a *different subset* from the pair-restricted rates in the Control/Fault columns. The two are not interchangeable: the paired rates are the ones Δ and the McNemar test refer to.

5.2 NO FAULT HAS A DETECTABLE EFFECT ON OFFICIAL SUCCESS

Table 3 gives the primary result on native-evaluator pairs, and Figure 1 in Appendix G plots it. Reading the point estimates alone, the picture is mixed and slightly absurd: DOM omission is -6.7 pp, parameter errors -12.9 pp, but popups are +9.7 pp and timeouts +3.2 pp — the fault arms doing *better* than control. Those signs are not evidence that popups help. They are what sampling noise looks like at this sample size, and the exact McNemar tests say so: no fault reaches significance (0.219 at the smallest, for parameter errors), and after Holm correction every adjusted p is exactly 1.000.

The bootstrap intervals tell the same story. The interval for parameter errors, the largest point estimate, spans $-12.9 \text{ pp} \in [-29.0, +0.0] \text{ pp}$ — a range that includes both a substantial degradation and no effect at all. Reporting “parameter errors do not significantly hurt the agent” would be reporting the width of that interval, not a property of the agent.

For completeness, Table 6 in Appendix B reports the same analysis with the evaluator paths pooled. It is not a robustness result and we do not use it as one; we include it to show how much the conclusion moves when the fallback contamination is left in.

5.3 THE DESIGN COULD NOT HAVE DETECTED A MODERATE EFFECT

A null on its own is uninformative; what makes these nulls interpretable is the power analysis. With 30–31 pairs per fault and an observed discordance of 0.234, the paired design cannot reach 0.80 power at *any* effect size it can express — the minimum detectable effect is undefined, not merely large. At the observed discordance, detecting a 10 pp difference would require on the order of 181 pairs per fault, roughly *six times* the 31 we have. Even a counterfactual $n = 40$ design reaches only 19.2 pp. The full power table, including the sample sizes implied by other discordances, is Table 5 in Appendix A.

The methodological consequence is that this family of null results is the default outcome at realistic WebArena sample sizes, and should be reported as an underpowered design rather than as evidence of robustness.

Table 4: Paired behavioral cost from the behavioural matrix (400 pairs, each logged twice). **Behavior-only**: no HAR was retained, so no row carries an official success verdict. Deltas are within-pair (fault – control); p is the two-sided Wilcoxon signed-rank test.

Fault	n pairs	Δ steps (mean)	Δ steps (HL)	p	Δ time (s, mean)	Δ time (s, HL)	p
DOM omission	80	−0.10	+0.0	0.956	−66.1	−46.0	0.068
Popup block	80	−0.44	+0.0	0.509	−27.5	−21.8	0.004
HTTP error	80	+4.22	+4.0	<0.001	+149.9	+132.3	<0.001
Timeout	80	−0.72	−0.5	0.021	−13.2	−8.9	0.316
Parameter error	80	+2.30	+1.5	<0.001	+20.6	+7.1	0.460

5.4 THE SAME FAULTS ARE SHARPLY VISIBLE AT THE PROCESS LEVEL

Table 4 turns to the behavioural matrix, where the same five faults are applied to 16 tasks over 5 seeds, giving 400 pairs with step and timing instrumentation. Here the effects are unambiguous, and they line up with mechanism rather than with layer:

- **HTTP errors** add +4.22 ReAct steps and +149.9 s per pair (both $p < 0.001$). An agent that encounters a server error re-navigates and retries; the cost is real and consistent.
- **Parameter errors** add +2.30 steps ($p < 0.001$): a corrupted argument costs a wasted action, the agent re-issues it, and the task usually still completes — which is exactly why the success-rate effect is hard to see.
- **DOM omission** and **popup blocking** make the agent *faster*, not slower (−66.1 s and −27.5 s; the latter at $p = 0.004$). We read this as fault-induced early termination rather than as a benefit: an agent that cannot see the page it needs gives up sooner.

Two cautions are built into how we report this. First, these rows carry no official verdict — no HAR was retained for this batch — so “cost” and “success” in this paper come from different experiments over different task sets, and we never combine them into a statement about cost per successful task. Second, absolute cross-arm timings are not interpretable: the five per-fault control arms are the same configuration, yet their mean wall-clock spans 103.7–224.6 s, a factor of 2.17. Only *within-pair* deltas, which share the machine state, are reported.

5.5 THE ARCHITECTURE PILOT CANNOT RANK ARCHITECTURES

Table 7 in Appendix C gives the only architecture comparison available. It is a pilot: $n = 9$ per cell, three tasks, and — we established by inspection — the committed runner does not reproduce these rows, so the artifact and the script that supposedly produced it disagree. We report it for completeness with two observations that survive even at this size.

First, the coarse `condition` column in the artifact pools *both* injected faults into a single 18-row bucket; grouping on it produces a “fault” row that is really two different faults averaged together, which we avoid by breaking rows out by fault. Second, `plan-and-execute` succeeds in 0.0% of 9 control trials against 44.4% for ReAct, while taking 364.8 s against 77.4 s — a $4.7\times$ wall-clock difference. A comparison at equal *step* budgets is therefore not a comparison at equal *time* budgets, and the artifact records no per-role call counters, so the two arms cannot be shown to have had equal LLM budgets. No architecture ranking follows from this, in either direction.

5.6 THE RECOVERY ARTIFACT IS DEGENERATE

The recovery artifact is the paper’s second methodological warning, with the column-by-column detail in Appendix D. It records behavioural flags — whether the observation was refreshed, whether the tool changed, whether a stale action was repeated — which is exactly the process-level evidence one would want for attribution. In practice 6 of its columns are constant across all 45 rows and carry no information, `plan_revised` was never recorded at all, and `recovered` is row-for-row identical to `official_success`: not an independent measurement of recovery, but a restatement of the outcome it would be used to explain.

We report it rather than silently dropping it because the failure mode is instructive: a recovery-behaviour artifact can look rich — a dozen behavioural columns — while containing two varying quantities, one of which is the outcome. Process-level attribution needs columns that vary *independently* of the outcome, and that has to be checked before the analysis, not after.

6 REPAIRS: DESIGNED, NOT YET EVALUATED

The motivating goal was to close the loop: attribute failures, apply targeted repairs, show a before/after improvement. **We have not run that ablation**, and nothing here should be read as a measurement.

The behavioural results do say where to aim. HTTP errors and parameter errors are *recoverable* faults — the success rate barely moves because the agent eventually gets there, at a cost of +4.22 steps and +149.9 s per pair for the former, and +2.30 steps for the latter — so a repair targeting either would be judged on process cost, which given section 5.3 is the only measure with the power to detect it at this scale. Repair evaluation should therefore inherit the paired, process-level protocol, not the success-rate one. Appendix F lists four candidate repairs that follow from these mechanisms, the one fix that *was* made during this work and why it improves the *measurement* rather than the *agent*, and what a credible before/after ablation would have to satisfy.

7 DISCUSSION

7.1 WHAT THIS DESIGN CAN AND CANNOT IDENTIFY

The question that motivated this work — how much of a web agent’s robustness is attributable to the model versus to the architecture — cannot be answered by these data, and we want to be precise about why rather than gesturing at “small n .”

Identifying a model effect requires the model to vary. In the official subset and the behavioural matrix the model is constant; in the architecture pilot it is also constant. A variance decomposition over these factors would partition variance into a term that is zero by construction, and any numeric “share” it produced would be an artefact of which factor happened to be varied. The same argument applies to the model $\times$ architecture interaction.

What *is* identified here is the fault effect, within one (model, architecture) configuration, and — more usefully — the properties of the measurement. Both of those are estimated with a paired design and reported with the power to interpret them.

The design that would identify the attribution question is a crossed one: Model $\times$ Architecture $\times$ Fault, with the same paired control/fault structure inside each cell, evaluated natively. Two models and two architectures over the five faults on the same 8 tasks would be enough to estimate main effects and the interaction; more models would be needed to separate a model effect from an idiosyncratic one. We give the concrete staged plan, with the measured per-trial wall-clock that sizes it, in the supplementary experiment plan released with this paper.

7.2 WHY THE NULLS ARE THE FINDING

It would be easy to read section 5.2 as a failed experiment and either bury it or explain it away. We argue the opposite: the nulls are reproducible and they have a structural cause. The success outcome is binary, the tasks are heterogeneous, the per-fault discordance is only 0.234, and the plausible effect sizes (a few percentage points) are far below what the design can resolve. Any underpowered fault-injection study on WebArena will produce this pattern, and because the pattern admits a flattering reading — “no significant degradation, therefore robust” — it is likely to be published as a positive result. Naming the failure mode seems more useful than adding one more underpowered table.

The constructive version of the same point is that the process measures *do* have power here. Step deltas of 2–4 are large relative to their within-pair variance, and they are significant at $n = 80$ pairs where the binary outcome is not. If robustness is the object of study, the measurement should be chosen with that in mind.

7.3 THREATS TO VALIDITY

Instrumentation. The dominant threat is the evaluator fallback (section 4.2), which we handle by stratification rather than by correction. A second risk is that the native path may itself mis-score tasks in ways we have not audited; the behavioural matrix, which has no official verdict at all, is reported separately for exactly this reason.

Synthetic faults. Our faults are injected at a fixed step by a seeded injector: deterministic and reproducible, which is what makes the paired analysis possible, but not draws from a real fault distribution. The injection *step* in particular is a free parameter we held fixed, and an agent’s robustness plausibly depends on when the fault arrives. This is the confound a reviewer should press on hardest, and the first thing our staged plan adds.

Scope. The behavioural matrix uses 16 tasks and 5 seeds — a larger and different task set than the 8-task official subset, and not joinable to it. The process results therefore generalise over more tasks than the success results do. As shown in section 5.6, the recovery artifact is degenerate and we draw no recovery conclusions from it; a credible analysis would need newly defined, outcome-independent labels, which is a contribution in its own right. Finally, one model, one architecture, one temperature, one provider: our claims are about the measurement protocol and this configuration’s fault sensitivity, not about web agents in general.

8 CONCLUSION

We ran a paired fault-injection study of a LangGraph ReAct agent on WebArena under the official evaluator, and its most useful product is a negative result about the measurement rather than about the agent.

End-to-end success is a blunt, high-variance instrument. With 30–31 native pairs per fault and an observed discordance of 0.234, no minimum detectable effect is defined at 0.80 power, no fault is significant after Holm correction, and detecting a 10 pp difference would need on the order of 181 pairs per fault. The same faults are unambiguous at the process level — HTTP errors alone add +4.22 steps and +149.9 s per pair — which is where robustness measurement should look. Layered on top of that is a warning about instruments: a compatibility fallback failed by construction on all 76 trials it handled and shifted the frozen baseline’s headline rate by 32.9 pp, more than any effect we set out to measure, while the recovery artifact consisted mostly of constant columns and an outcome restated as a behavioural label. Neither would have been visible from a single success number.

The model-versus-architecture question remains open: answering it requires a crossed design under the same paired protocol, and we release the staged plan for it alongside this paper. What we can say now is narrower and, we think, more durable: for web agents, report the paired design, stratify the evaluator, state the power, and measure the process — because the success rate alone will not tell you whether your agent is robust or merely unmeasured.

REPRODUCIBILITY STATEMENT

Every number in this paper is generated by one command, `python3 scripts/paper_tables.py`, from the committed artifacts under `experiments/`. The prose does not contain literals: it references macros that the same command writes, so regenerating the pipeline rewrites both the tables and the numbers the text points at. The generator emits a per-quantity audit trail to `experiments/analysis/paper_v1/stats/`, including the concordance counts (a, b, c, d) behind every test, both bootstrap intervals, and a provenance table recording which artifact produced which row and whether it may enter an official-success denominator.

The statistical routines are implemented against the Python standard library only, with no SciPy or NumPy dependency, in `scripts/paper/stats_core.py`: exact and mid- p McNemar tests from the exact binomial tail, Wilson intervals, a percentile bootstrap over pair units, Wilcoxon signed-rank (exact enumeration for small n , tie-corrected normal approximation above), Hodges–Lehmann estimation, Holm and Benjamini–Hochberg adjustment, and the paired-proportion minimum detectable effect. These are unit-tested against hand-computed values.

Bootstrap intervals are reported with a fixed seed (`BootSeed=20260914`, 20000 resamples) and are additionally computed under five further seeds; the envelope is in the released artifacts, so a reader can see how much of any interval is a seed artefact.

Three limitations on reproducibility we state rather than paper over. The underlying language-model provider does not guarantee bit-identical sampling for a fixed seed, so the experiments are reproducible at the level of the design (identical fault, injection step, task, and seed pairing) rather than the token sequence. Wall-clock measurements are host- and load-dependent, which is why only within-pair deltas are reported. And the architecture pilot’s committed rows are not reproducible by the committed runner — we report that as a finding (section 5.5) rather than presenting the rows as reproducible.

Raw traces, HAR archives, browser authentication state, and credentials are excluded from the repository by `.gitignore` and are not part of this submission; the aggregated artifacts the pipeline reads are. Credentials are read from the environment only and are never committed.

ETHICS STATEMENT

This work studies the reliability of autonomous web agents and therefore touches on the risks of deploying them.

Automated browsing. All experiments ran against self-hosted WebArena replicas with synthetic data on isolated infrastructure. No live third-party site was traversed, no real account was used, and no external service was affected. The authentication state used by the agent exists only inside that sandbox.

Fault injection. The injected faults — dropped DOM nodes, blocking overlays, HTTP errors, timeouts, and corrupted tool arguments — are non-adversarial: they degrade the agent’s own inputs and do not target, probe, or exfiltrate anything. The injection harness is deterministic and seeded, and it is released so that its effects can be reproduced rather than speculated about.

Dual use. A systematic account of which faults a web agent fails to notice could in principle inform an attacker seeking to induce failures, or a site operator seeking to block agents. We judge the risk low relative to the benefit: the faults are already ubiquitous in ordinary web operation, the findings are mostly about measurement rather than about exploitable weaknesses, and the mitigation we argue for — process-level monitoring and evaluator hygiene — helps defenders. We release no attack tooling and no adversarial payloads.

Evaluation integrity. The paper’s central methodological claim is that a widely used evaluation setup can be wrong in a way that flatters the system being measured, and that a structurally failure-biased evaluator path went undetected. We report this because it affects how published numbers should be read, not to single out a particular benchmark; the failure is in the reporting convention, not in the benchmark’s design.

AI USE STATEMENT

A large language model was used in the preparation of this work, and we describe its role precisely.

Where it was used. (i) Writing and editing the manuscript, including restructuring, translation, and prose revision. (ii) Writing the analysis code in `scripts/paper/` — the statistics module, the artifact loaders, and the table generator. (iii) Auditing the committed experiment artifacts, where it surfaced the evaluator-fallback contamination, the degenerate recovery columns, the 800-record matrix being 400 pairs, and the architecture table’s pooled fault bucket.

Where it was not used. The fault-injection harness, the agent implementation, and the experiment runs themselves predate this assistance. The model did not design the experiments, select the tasks, or produce any experimental result. It did not fabricate, interpolate, or adjust any number: every quantity in the paper is computed by `scripts/paper_tables.py` from the committed artifacts, and that script is released.

Verification. Two safeguards apply to the analysis code, which is the part where an error would silently change a result. First, the statistical routines carry unit tests with hand-computed expected values, and one such test caught a real defect during preparation —

an incorrect search bracket in the minimum detectable effect calculation that made the function return an undefined value for every input. Second, the main tables are checked against independently committed summaries: the pooled paired success rates reproduce experiments/analysis/official_subset_v1_final/official_summary.csv cell for cell. The author reviewed and verified all generated output and takes responsibility for the content.

REFERENCES

- Ivan Evtimov, Minghao Zhuge, Andrea Caforio, et al. WASP: Benchmarking web agent security against prompt injection attacks. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2504.18575>.
- Aarav Gupta et al. ReliabilityBench: Evaluating consistency, robustness, and fault tolerance of LLM agents. *arXiv preprint*, 2025.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://arxiv.org/abs/2401.13649>.
- Ido Levy et al. SecureWebArena: A comprehensive benchmark for web agent security. *arXiv preprint*, 2025.
- LangChain Team. LangGraph: Build stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>, 2024. Accessed 2026.
- Chen Wang et al. MAS-FIRE: A systematic fault injection and attribution framework for LLM-based multi-agent systems. *arXiv preprint*, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Yifan Zhang et al. ChaosLLM: Estimating the reliability of LLM agents under non-adversarial system faults. *arXiv preprint*, 2024.
- Yu Zhang et al. WAREX: A reliability evaluation tool for web agents via transparent network fault injection. *arXiv preprint*, 2025.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2307.13854>.

A STATISTICAL PROCEDURE IN FULL

Success rate. The primary test is the two-sided *exact* McNemar (binomial) test on the discordant counts, computed from the exact binomial tail rather than the χ^2 approximation, because the discordant counts here are small ($b + c \leq 10$ per fault). We also report the mid- p variant, which is less conservative at these sizes, and the Haldane–Anscombe corrected matched odds ratio. Multiplicity across the five faults is controlled by Holm–Bonferroni; Benjamini–Hochberg is implemented and reported in the released code for comparison.

Intervals. Confidence intervals for $D(F)$ come from a percentile bootstrap that resamples *pair units* — the correct resampling unit for a paired design, as opposed to resampling trials, and the seed is fixed (BootSeed=20260914, 20000 resamples) so the interval is reproducible. Because a single bootstrap seed can be selected to flatter a result, we also compute the interval under five additional seeds and report the envelope in the released artifacts. We additionally computed a task-clustered sensitivity interval, which is wider than the pair-level interval when the same task is correlated across seeds; both are in the released statistics files.

Table 5: Minimum detectable paired difference in success rate (percentage points) at $\alpha = 0.05$, power 0.80. **n/a** means the design cannot reach the target power at *any* effect size the paired design can express (the risk difference is bounded by π_d). The native-evaluator analysis has only $n = 30$ –31 pairs per fault. The $n = 40$ column is therefore a counterfactual, and a null result under the actual design cannot be read as evidence of robustness.

Discordance π_d	MDE at $n = 31$	MDE at $n = 40$	MDE at $n = 80$	MDE at $n = 155$
0.10	n/a	n/a	9.8	7.1
0.15	n/a	n/a	12.0	8.6
0.20	n/a	19.2	13.8	10.0
0.25	24.2	21.5	15.4	11.2
0.30	26.5	23.6	16.9	12.2

Table 6: Primary fault result, *all* evaluator paths pooled. Reported for completeness; the fallback adapter is structurally failure-biased (see Table 3).

Fault	n	Disc.	Control	Fault	Δ (pp)	95% CI (pp)	p_{exact}	p_{mid}	p_{Holm}	OR
DOM omission	40	15	67.5%	65.0%	−2.5	[−22.5, +17.5]	1.000	0.804	1.000	0.88
Popup block	40	11	57.5%	60.0%	+2.5	[−12.5, +17.5]	1.000	0.774	1.000	1.18
HTTP error	40	12	57.5%	62.5%	+5.0	[−12.5, +22.5]	0.774	0.581	1.000	1.36
Timeout	40	6	65.0%	65.0%	+0.0	[−12.5, +12.5]	1.000	1.000	1.000	1.00
Parameter error	40	8	77.5%	62.5%	−15.0	[−27.5, −2.5]	0.070	0.039	0.352	0.20
All faults	200	52	65.0%	63.0%	−2.0	[−9.0, +5.0]	0.678	0.583	1.000	0.86

Δ is the paired risk difference (fault − control); the interval is a percentile bootstrap over pair units and is descriptive — the exact McNemar p is the primary inferential quantity. OR is the Haldane-corrected matched odds ratio.

Process measures. Deltas in step count and wall-clock time are not binary, so we use the two-sided Wilcoxon signed-rank test with the Hodges–Lehmann estimator of the shift; the distributions are heavy-tailed and contain ties, which is why we do not report a paired t -test. These tests are run on the behavioural matrix, which is *not* the same data as the official subset (section 4).

Power. The minimum detectable effect for a paired proportion design is obtained by solving

$$n = \frac{(z_{1-\alpha/2}\sqrt{\pi_d} + z_{\text{power}}\sqrt{\pi_d - \delta^2})^2}{\delta^2} \quad (2)$$

for δ , where $\pi_d = \mathbb{P}(\text{arms disagree})$. Two subtleties we had to get right, and that we suspect are easy to get wrong: the risk difference is bounded by π_d , not by 1, so the search must be bracketed on $(0, \pi_d]$; and when the bracket is wrong the equation silently returns undefined rather than failing loudly. We use the *observed* discordance $\pi_d = 0.234$ rather than a round number, and we report cases where the target power is unreachable at any effect size as *undefined* rather than quoting a number.

B POOLED-EVALUATOR TABLE

For completeness, Table 6 reports the same analysis with the evaluator paths pooled. It is not a robustness result and we do not use it as one; we include it to show how much the conclusion moves when the fallback contamination is left in, and to demonstrate why any published number of this form should be accompanied by its stratified version.

C ARCHITECTURE PILOT TABLE

Table 7 gives the raw architecture pilot discussed in section 5.5. Rows are broken out by fault rather than by the artifact’s coarse `condition` column, which pools two different faults into one bucket.

Table 7: Architecture comparison — **PILOT**, $n = 9$ **per cell**. Rows are broken out by fault; *not* by the artifact’s coarse condition column, which pools the 2 injected faults into a single 18-row bucket. The committed `run_architecture_matrix.py` does not reproduce these 54 rows (different task set and seed range), so this table supports no architecture ranking. The two architectures also differ sharply in wall-clock (77.4 s vs 364.8 s on the control arm), which confounds any equal-budget comparison; the artifact records no per-role call counters, so budget equality cannot be verified.

Architecture	Condition	n	Success	Rate	Steps	LLM calls	Time (s)
plan_execute	none (control)	9	0/9	0.0%	3.11	6.89	364.8
plan_execute	web_dom_missing	9	1/9	11.1%	4.22	4.22	516.4
plan_execute	web_http_error	9	1/9	11.1%	5.22	5.22	788.5
react	none (control)	9	4/9	44.4%	5.00	5.00	77.4
react	web_dom_missing	9	4/9	44.4%	4.89	4.89	82.6
react	web_http_error	9	3/9	33.3%	8.00	8.00	218.9

Table 8: The recovery artifact on its 45-trial subset. It is **degenerate**: `observation_after_fault`, `plan_revised`, `premature_stop`, `retry_executed`, `stale_action_repeated`, `tool_changed` are constant across all 45 rows and therefore carry no information; `plan_revised` was never recorded (empty, not false); and `recovered` is row-for-row identical to `official_success`. `behavior_labels` is a deterministic function of `recovered` (`recovered` $\Rightarrow$ `observation_refreshed`, otherwise `fault_detected`), not an independent observation. Only the columns shown vary.

Fault	n	Official success	Rate	goto_fallback
Parameter error	9	8/9	88.9%	0/9
DOM omission	9	7/9	77.8%	0/9
HTTP error	9	6/9	66.7%	2/9
Popup block	9	8/9	88.9%	0/9
Timeout	9	7/9	77.8%	0/9
All faults	45	36/45	80.0%	2/45

D THE RECOVERY ARTIFACT IN FULL

The recovery artifact records behavioural flags — whether the observation was refreshed, whether the tool changed, whether a stale action was repeated — which is exactly the process-level evidence one would want for attribution. In practice 6 of its columns are constant across all 45 rows and carry no information; `plan_revised` was never recorded at all; and `recovered` is row-for-row identical to `official_success`, so it is not an independent measurement of recovery but a restatement of the outcome it would be used to explain. The `behavior_labels` column is a deterministic function of `recovered` (`recovered` $\Rightarrow$ `observation_refreshed`, otherwise `fault_detected`), so correlations between the two are tautological. Two further columns are the same predicate under different names (`stale_action_repeated` and `retry`), and `fault_detected` is recorded only on the not-recovered subset, so it is computed over a different denominator from the rest of the table.

We report the table rather than silently dropping it because the failure mode is instructive: a recovery-behaviour artifact can look rich — a dozen behavioural columns — while containing two varying quantities, one of which is the outcome. Process-level attribution needs columns that vary *independently* of the outcome, and that has to be checked before the analysis, not after.

E REPRODUCIBILITY OF THE RUNS

Every trial records its fault seed, injection step, model profile, architecture, and git commit. The provider does not guarantee bit-identical sampling for a fixed seed, so our reproducibility claim is at the level of the design (same fault, same step, same task/seed pairing) rather than the token sequence; we state this explicitly rather than claiming seed determinism we cannot verify. Wall-clock is host- and load-dependent, which is why process-cost claims are made on within-pair deltas only — the five

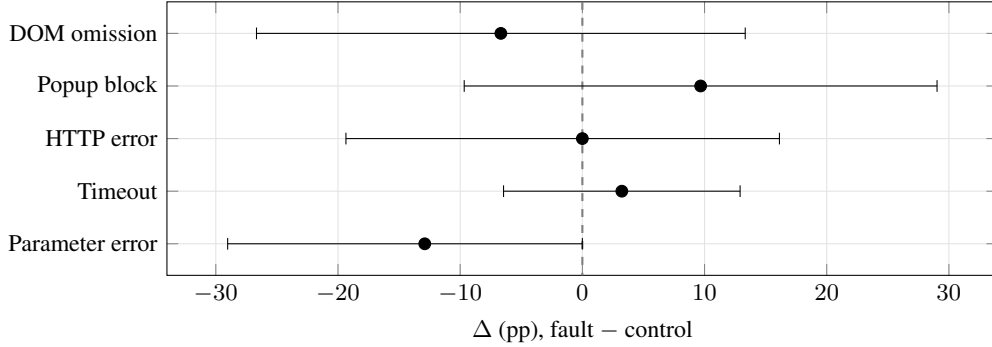


Figure 1: Paired risk difference Δ in official success (fault – control) on native-evaluator pairs, with the percentile bootstrap interval over pair units. Every interval spans zero and no fault survives Holm correction, which is the paper’s primary result. Compare Table 3.

per-fault control arms are the *same* configuration yet differ by a factor of 2.17 in mean wall-clock (103.7–224.6 s), so cross-arm absolute timings are dominated by machine-level drift and are not interpretable.

F CANDIDATE REPAIRS AND WHAT AN ABLATION WOULD NEED

Four policy changes follow from the mechanisms in section 6: error-aware retry with backoff instead of re-deriving the whole plan; argument validation against the current observation before dispatch; stale-observation detection that forces a re-observation; and a termination policy for unobservable state. The last is a judgement call rather than an automatic improvement — the DOM-omission and popup arms are *faster* than control (–66.1 s and –27.5 s), which we read as early termination, so the objective could be either to shorten that path further or to have the agent report the blockage instead of silently failing. Each is testable in the existing harness by running the same paired design with the repair enabled in both arms. None has been run.

One fix *was* made during this work — correcting the task schema and completion check, and detecting the compatibility fallback contaminating the evaluator (section 4.2) — and it must be classified correctly: it raises the measured success rate by up to 32.9 pp, more than any fault effect in this paper, and it improves the *measurement*, not the *agent*. Reporting it as a robustness improvement would be precisely the error this paper is about; any web-agent robustness claim drawn from a pipeline with a fallback path, a schema mismatch, or an unvalidated completion check is measuring the harness.

A credible before/after comparison would therefore need to (i) re-use the paired design and the native-only stratum, (ii) be sized against section 5.3 rather than against available compute — at the observed discordance a 10 pp success-rate improvement needs roughly 181 pairs per arm, so a process-cost improvement is the realistic target at smaller n — and (iii) run the repaired arm concurrently with the unrepaired one, so that machine-level drift, which spans a factor of 2.17 across nominally identical arms, does not masquerade as an effect.

G GRAPHICAL SUMMARIES

Figures 1, 2 and 3 show the three quantitative summaries in graphical form. They are generated by the same command as the tables, from the same statistics, so they cannot drift from them.

H SUPPLEMENTARY EXPERIMENT PLAN

The staged design that would make the attribution question answerable is released with this paper. It is ordered by its ability to change the numbers reported here, not by novelty. **Stage A** first repairs the evaluator’s null-retrieval schema path and re-evaluates the existing 400 response/HAR pairs

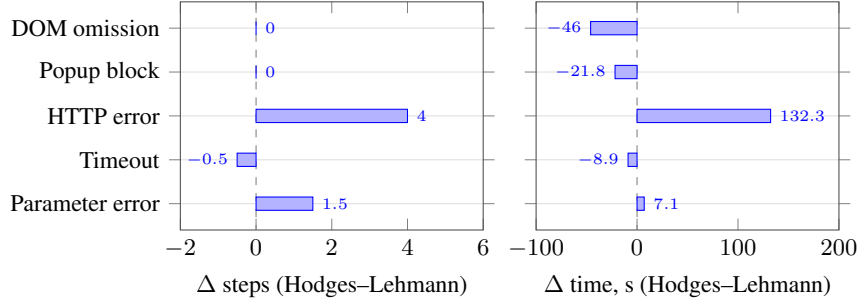


Figure 2: Within-pair process cost on the behavioural matrix. HTTP errors and parameter errors cost the agent steps and time, while DOM omission and popup blocking make it *faster* — which we read as fault-induced early termination. All five are significant at $p < 0.01$ except where noted in Table 4.

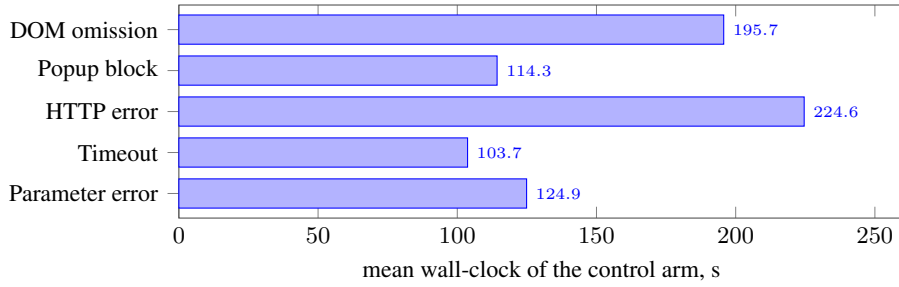


Figure 3: The noise floor. These five arms are the *same* agent configuration under the clean condition; each is the control half of one fault’s pair set. Their means span a factor of $2.17\times$, so absolute cross-arm timings are dominated by machine-level drift and only within-pair deltas are interpretable.

offline. Every frozen row already retains HAR, so repeating the same agent trials would not remove the schema-triggered fallback; new runs are needed only for artifacts that the repaired evaluator cannot consume or for deliberately replaced tasks. **Stage C** crosses three categorical model levels with three architectures and three faults. We do not impose an ordinal model-capability contrast: current provider claims invalidate the earlier Flash-Pro ordering, and the cross-provider ordering is not independently identified. The confirmatory target is therefore the omnibus model $\times$ architecture interaction followed by preregistered pairwise and simple-effect contrasts. **Stages E and F** add injection-step sensitivity and the repair ablation, respectively; the former Stage B and D expansions are folded into Stage C. Injection-step sensitivity is the confound we would press on hardest as a reviewer, since our injector holds the step fixed throughout. The plan gives exact command lines, the six-worker wall-clock budget measured from the runs reported here, and the per-trial timings that size each stage.